



# Mastering the Page Object Model

---

# The Necessity of a Robust Framework

Building a test automation framework is not merely about writing scripts; it is about creating a sustainable ecosystem for quality assurance.



## Achieve Uniformity

Standardises coding practices across the team.



## Enhance Reusability

Write code once, use it everywhere.



## Increase Productivity

Reduces time spent on fixing flaky tests.



## Simplify Maintenance

The primary goal. If the UI changes, updates should be localised, not systemic.



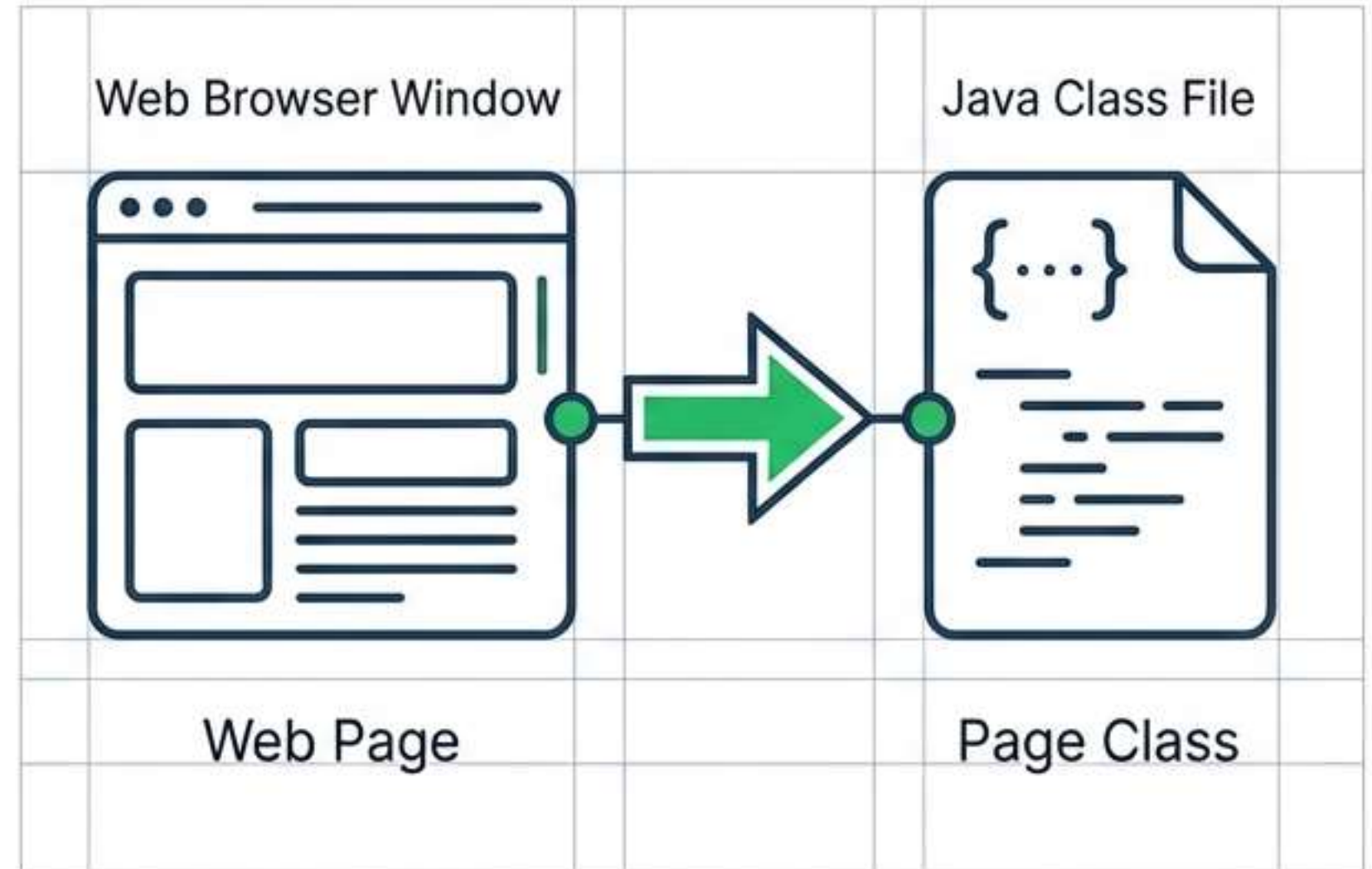
# What is the Page Object Model (POM)?

**Page Object Model** is a widely adopted design pattern in test automation that creates an **Object Repository** for web UI elements.

**The Concept:** Each web page in your application is represented by a corresponding **Class** in your code.

**The Function:** This class serves as an interface to the page of your application, hiding the implementation details.

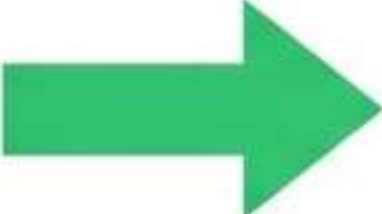
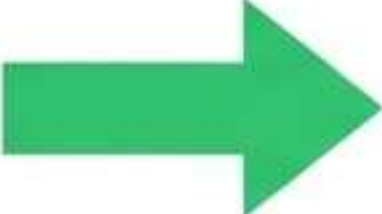
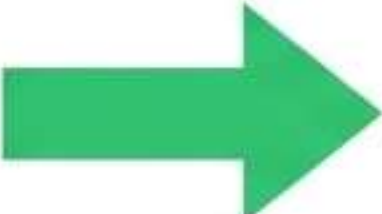
**The Ecosystem:** POM forms the backbone of hybrid frameworks utilizing **Selenium**, **TestNG**, **Apache POI**, and BDD tools.





# What is the Page Object Model (POM)?

Translating User Interface components into **Object-Oriented** structures.

| Web Context                           | Transition                                                                            | Code Context                                  |
|---------------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------|
| The Web Page                          |    | The <b>Java Class</b>                         |
| Web Elements<br>(Text boxes, Buttons) |   | <b>Class Variables</b><br>(Properties/Fields) |
| User Actions<br>(Clicking, Typing)    |  | <b>Class Methods</b><br>(Functions)           |

*"The Page Object encapsulates the page's implementation details,  
exposing only the services the page offers."*

# Strategic Benefits of POM

## Decoupling

Separates Test Logic (Validation) from Technical Implementation (Locators and Actions).

## Reduced Redundancy

If a locator changes, fix it in one Class file, not in fifty different test scripts.

## Flexibility

Supports multilingual testing and data-driven approaches seamlessly.

## Code Cleanliness

Results in test scripts that are readable, concise, and focused on business logic.



# Architectural Separation of Concerns

## The Test Layer (Client)

- Contains @Test annotations
- Holds Assertions & Validation logic
- Rule: NEVER contains HTML locators or WebDriver calls.



## The Page Layer (Service)

- Contains Page Factory / By locators
- Performs Actions (e.g. enterUsername)
- Rule: NEVER contains assertions.

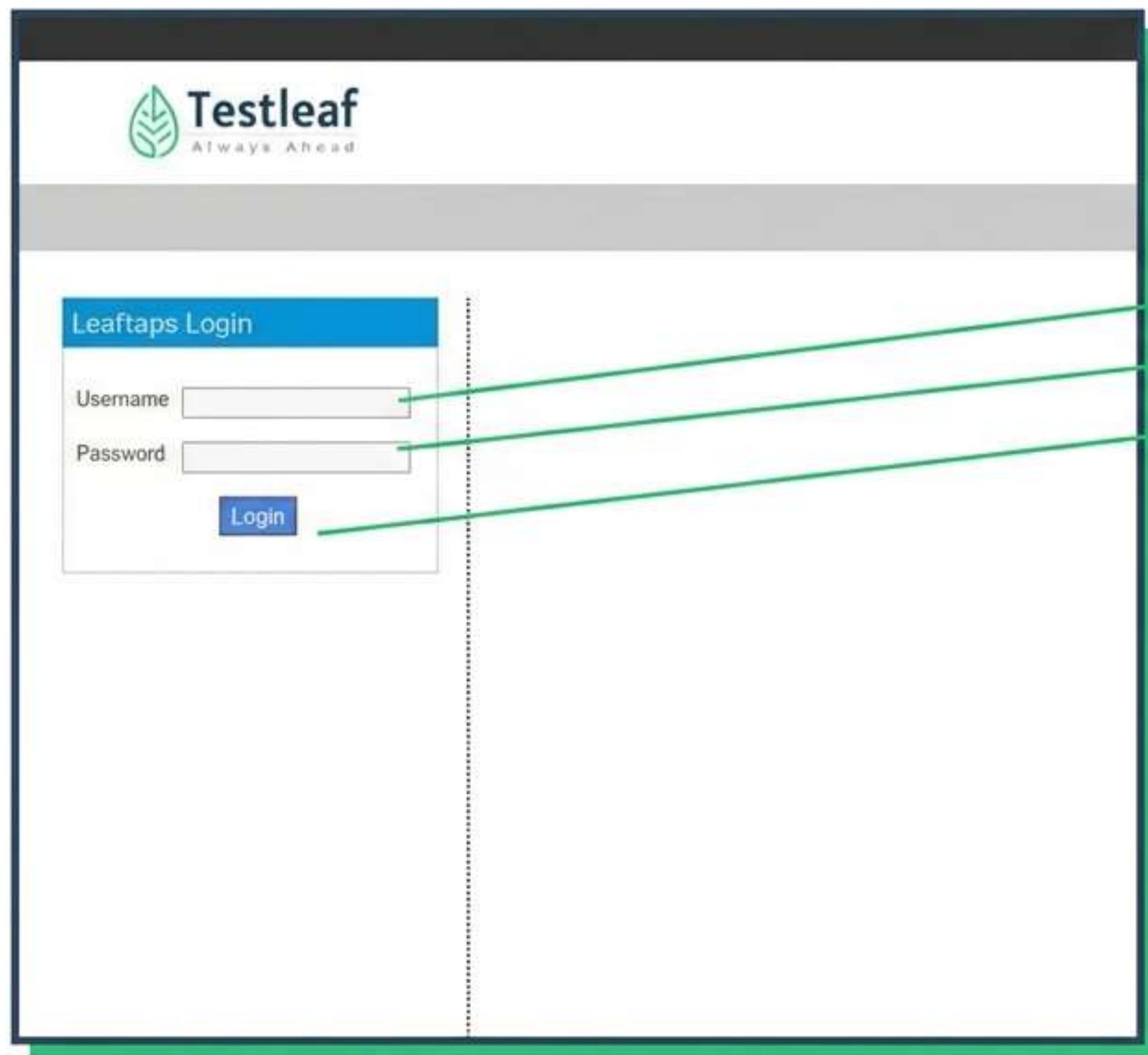
# Practical Scenario: The Lead Creation Journey

Applying POM logic to the 'LeafTaps' application workflow.





# Phase 1: The Login Interaction



The screenshot shows the Testleaf login page. At the top left is the Testleaf logo with the tagline 'Always Ahead'. Below it is a blue header bar. The main content area contains a 'Leaftaps Login' form. The form has a blue title bar, followed by 'Username' and 'Password' labels next to input fields, and a blue 'Login' button. Three green lines with arrowheads point from the code on the right to the form: the first points to the 'Username' input field, the second points to the 'Password' input field, and the third points to the 'Login' button.

```
public class LoginPage {
```

```
// Elements (Properties)
```

```
WebElement username;
```

```
WebElement password;
```

```
WebElement loginButton;
```

```
// Actions (Methods)
```

```
public void enterUserName(String data) { ... }
```

```
public void enterPassword(String data) { ... }
```

```
public void clickLogin() { ... }
```

```
}
```



# Page Chaining & Navigation Logic

POM supports a fluid coding style that models the natural navigation of the application.



```
public HomePage clickLogin() {  
    driver.findElement(By.id("login")).click();  
    return new HomePage(); // Returns the next page object  
}
```

# Phase 2: Form Interaction (Create Lead)

My Home **Leads** Contacts Accounts Cases Activities Opportunities Quotes Orders Forecasts Marketing Partners Team Openings

Last Viewed: sm vd (Microsoft) | sm vd (Microsoft) | Alreys Serna (hit) | Alreys Serna (hit) | Alreys Serna (hit) | Alreys Serna (hit) | Alreys Serna (hit)

Shortcuts

- My Leads
- Creates Lead**
- Red Leads
- Merge Leads
- Upload Leads
- Request Csting

Create Account

Create Contact

Create Case

Create Lead

Create Opportunity

## Create Lead

1 Company Name

2 Forename

Sources

First Name (Local)

Salutation

Title

Annual Revenue

Industry

Ownership

SIC Code

Description

Important Note

Parent Account

3 Surname

Marketing Campaign

Last Name (Local)

Date of birth

Department

Preferred Currency

Number of Employees

Ticker Symbol

```
public class CreateLeadPage {
```

```
// Actions map to Form Fields
```

```
1 public CreateLeadPage enterCompanyName(String name) { ... }
```

```
2 public CreateLeadPage enterFirstName(String fName) { ... }
```

```
3 public CreateLeadPage enterLastName(String lName) { ... }
```

```
// Navigation Action
```

```
public ViewLeadPage clickSubmit() {
```

```
    submitButton.click();
```

```
    return new ViewLeadPage();
```

```
}
```

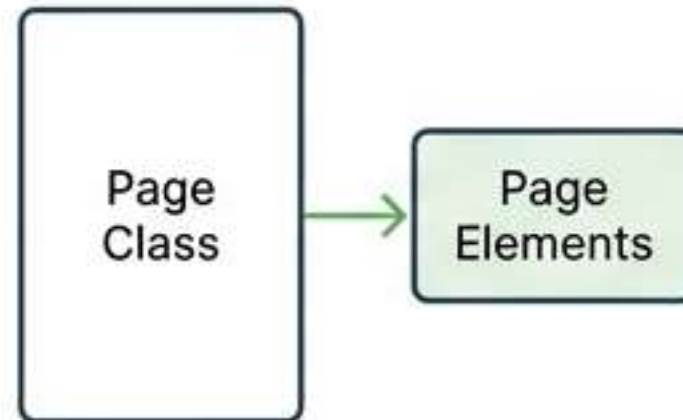
```
}
```



# Implementation Best Practices

## 1. Single Responsibility

Each page class should only deal with the elements of that specific page.



## 2. Naming Conventions

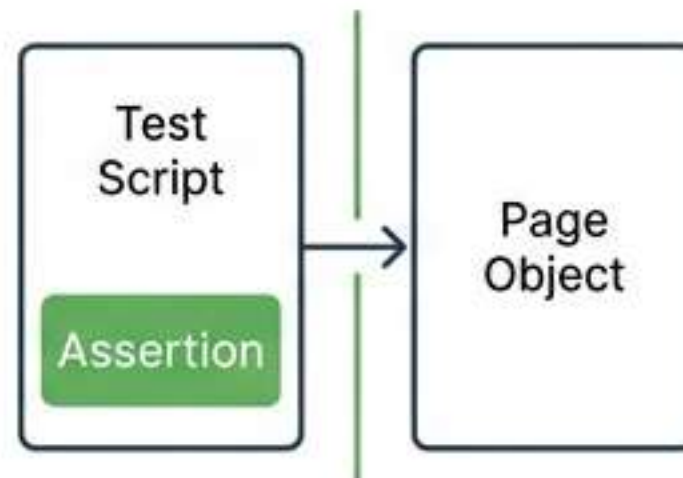
Use clear, descriptive names. "clickLoginButton" is superior to "doClick".

~~'doClick()'~~ ✗

'clickLoginButton()' ✓

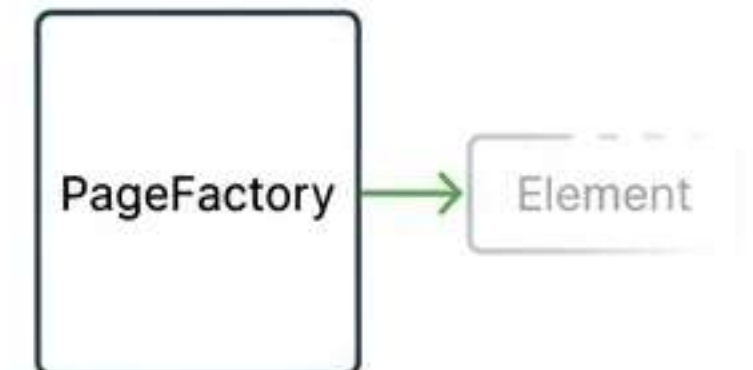
## 3. No Assertions in Pages

Keep validation in the Test scripts to ensure Page Objects remain reusable for different scenarios.



## 4. Lazy Initialization

Utilise PageFactory to initialise elements only when they are needed.



# Executive Summary

# 01

---

## Structure

POM organises code based on the UI structure, creating a logical object repository.

# 02

---

## Maintainability

It isolates changes. A UI update requires a fix in only one place, safeguarding the framework.

# 03

---

## Efficiency

Through page chaining and reusable components, it accelerates script development.